# *FULL PROJECT REPORT*

## *Submitted To,*

## *Mr. Muhammad Rizwan*


## *Submitted By,*

***Mohammad Rohaan 22I-2327 Sec-A***

***Ahmed Abdullah 21i-2539 Sec-A***


# ***Optimization of Sparse Matrix–Vector Multiplication (SpMV) in the HPCG Benchmark***

## ***1. Introduction***

High-performance computing (HPC) systems are widely used in scientific simulations, engineering applications, and large-scale numerical problems. One of the most important operations in scientific computing is solving large sparse linear systems. The **High Performance Conjugate Gradient (HPCG)** benchmark is a modern performance benchmark designed to evaluate how well a computer system performs real-world workloads rather than synthetic peak performance. Unlike the High-Performance Linpack (HPL), which focuses mainly on dense matrix operations, the HPCG benchmark stresses memory access, communication, and sparse computations that appear in actual scientific codes.

The main purpose of this project is to study the internal working of the HPCG benchmark and improve the performance of one of its main kernels. In this project, the **Sparse Matrix–Vector Multiplication (SpMV)** kernel was selected and optimized using a loop-unrolling strategy to improve computational efficiency.

## 2. Why HPCG Benchmark

HPCG was developed to represent real application behavior more accurately than traditional benchmarks. Real scientific workloads depend heavily on:

- Sparse matrix computations
- Memory bandwidth and latency
- Irregular memory access
- Multigrid solvers
- Global communication patterns

HPCG includes the following major kernels:

- Sparse Matrix–Vector Multiplication (SpMV)
- Symmetric Gauss–Seidel Smoother (SymGS)
- Dot Products
- Vector Updates (WAXPBY)
- Multigrid V-cycle

Among these, **SpMV is the most time-consuming kernel**, making it an ideal candidate for optimization.

## 3. Selected Kernel: Sparse Matrix–Vector Multiplication (SpMV)

SpMV computes the product of a sparse matrix and a vector:

$$y = A \times x$$

In sparse matrices, most elements are zero, so only nonzero values are stored and used. SpMV plays a critical role in iterative solvers such as the Conjugate Gradient method. Since SpMV involves irregular memory access, it is usually **limited by memory bandwidth rather than computation speed**.

Because of this, even small optimizations can lead to noticeable performance improvements.

## 4. Selected Research Paper and Inspiration

The optimization strategy for this project is inspired by research on improving SpMV through better use of CPU registers and loop efficiency. Techniques such as **loop unrolling, vectorization, and cache-aware memory access** are commonly used to accelerate SpMV on modern processors.

In this project, a **4-way loop unrolling strategy** was implemented to reduce loop overhead and improve instruction-level parallelism.

For this project, the primary reference chosen is the paper **"CVR: Efficient Vectorization of SpMV on x86 Processors"** by Biwei Xie et al., published at CGO 2018. The paper proposes a new sparse matrix format called **Compressed Vectorization-Oriented sparse Row (CVR)**, specifically designed to improve the vectorization and cache efficiency of sparse matrix–vector multiplication (SpMV) on modern x86 many-core processors such as Intel Xeon Phi (Knights Landing).

In addition, I also referred to the survey **"A Systematic Literature Survey of Sparse Matrix-Vector Multiplication"** by Gao et al., which provides a broad overview of existing SpMV optimization techniques, sparse formats (CSR, ELL, DIA, SELL, BCSR, CSR5, etc.), and architecture-specific optimizations on CPUs, GPUs, FPGAs and heterogeneous systems.

Although my implementation is much simpler than the full CVR method, this paper was selected because it clearly shows **why vectorization and memory access patterns are so important for SpMV**, and it focuses on **CPU-oriented optimizations**, which matches the environment used in this project (x86 CPU via WSL).

## 4.1 Short Summary of the CVR Paper

The CVR paper starts from the observation that SpMV is **memory-bound and irregular**, and that traditional CSR-based kernels struggle to exploit wide SIMD units and caches effectively.

The authors identify two major problems:

1. **Irregular access to the input vector x**
    - Elements of x are accessed using scattered column indices, which causes **poor cache locality** and many **L2 cache misses**.
2. **Inefficient reduction pattern**
    - Each output element y[i] is the sum over all non-zeros in row i, which leads to scalar accumulations and complicated reduction when using SIMD.

To address this, the paper proposes a new storage format **CVR (Compressed Vectorization-oriented sparse Row)**:

- The matrix rows are **re-arranged and packed into "columns" that match SIMD lanes**.
- Each SIMD lane is responsible for processing one row at a time.
- When a row finishes in one lane, that lane immediately picks up the next available row (using a tracker mechanism), balancing the work across lanes.
- The data layout keeps accesses to x relatively localized, which improves cache reuse.
- SpMV is then implemented directly on this CVR layout, leading to:
    - Better **cache locality**
    - Higher **SIMD utilization**
    - Lower **preprocessing overhead** compared to other advanced formats such as CSR5, ESB, and VHCC.

In their experiments on Intel Knights Landing with 58 sparse matrices (30 scale-free + 28 HPC), CVR achieves up to **1.70× (1.33× average) speedup** over the best existing SpMV implementation on scale-free matrices, and up to **1.57× (1.10× average)** on HPC matrices, while also showing **much lower preprocessing overhead**.

## 4.2 Reason for Choosing This Paper

I chose the CVR paper as my main reference for several reasons:

1. **Direct focus on SpMV on x86 CPUs**
    - The paper targets exactly the same kernel that I optimized in the HPCG code (SpMV), on the same kind of architecture (x86 CPU).

2. **Clear connection to vectorization and caches**
    - CVR is explicitly "vectorization-oriented". It systematically explains why **SIMD lanes**, **cache locality**, and **data layout** matter, which helped me understand why even a simple loop-level optimization (like loop unrolling) can bring improvement.

3. **Good bridge between theory and practice**
    - While I did not implement the full CVR format, the ideas in the paper guided my thinking:
        - Reduce overhead per non-zero
        - Make the compiler's job easier for SIMD
        - Try to access memory in more regular patterns

4. **Support from the SpMV Survey**
    - The systematic survey shows that there are many families of SpMV optimizations (format-based, auto-tuning, machine learning, mixed-precision, etc.), but for this project, a **lightweight CPU-side optimization** is more realistic than building a completely new format.

# 5. Critical Review of the Approach

## Strengths

- **High SIMD Utilization:**
  CVR maps each matrix row to a SIMD lane, allowing multiple rows to be processed in parallel. This leads to more efficient use of wide vector units such as AVX-512.
- **Better Cache Locality:**
  By carefully arranging non-zeros into CVR "columns", access to the input vector x becomes more localized. The paper shows that CVR significantly reduces L2 cache miss ratios compared to other formats for many scale-free matrices.
- **Low Preprocessing Overhead:**
  Many advanced formats for SpMV (e.g., CSR5, ESB, VHCC) suffer from heavy preprocessing cost. CVR is designed to keep the conversion overhead relatively low, which is important when the number of SpMV iterations is limited. The paper shows that CVR needs far fewer iterations to amortize preprocessing than some competing methods.
- **Robust to Matrix Irregularity:**
  CVR is designed to work well on both **HPC matrices** and **scale-free graphs**, which are highly irregular and sparse. This makes it more general than some formats that only perform well on regular stencil-like matrices.

## Weaknesses / Practical Limitations

- **Format Conversion Complexity:**
  Although CVR's preprocessing overhead is lower than some other advanced formats, it is still **much more complex** than the simple CSR format used in HPCG. Implementing full CVR inside the HPCG code base would require:
  - Designing a new data structure
  - Modifying matrix generation
- **Hardware-Specific Tuning:**
  The paper is evaluated primarily on Intel Xeon Phi (KNL) with AVX-512. Some of its ideas (lane count, memory modes, gather/scatter cost) are tuned for that platform. On a modest 2-core / 4-thread laptop CPU running under WSL, the full CVR design may not achieve the same relative gains.
-

- **Implementation Complexity vs. Educational Value:**
  For learning purposes, a **simpler kernel-level optimization** (such as loop unrolling in the innermost SpMV loop) is easier to understand, implement, debug, and explain in a report and presentation. Full CVR implementation would shift the project focus from "numerical computing and benchmarking" to "large-scale systems engineering".

## 6. Proposed Optimization Strategy

The original SpMV implementation used a **simple inner loop** that processes one nonzero element at a time:

```
for (int j = 0; j < cur_nnz; j++) {
    sum += cur_vals[j] * xv[cur_inds[j]];
}
```

This was replaced with a **4-way unrolled loop**:

- Four partial sums are computed in parallel.
- Fewer loop control instructions are executed.
- Better use of CPU registers.
- Reduced instruction overhead.

This approach improves **instruction-level parallelism (ILP)** and reduces pipeline stalls.

## 7. Implementation Details

The optimized implementation was applied in the file:

src/ComputeSPMV_ref.cpp

Changes were made only inside the **main SpMV loop**, without altering:

- Matrix format
- Data structures
- Input/output logic

This ensured that correctness and compatibility with the rest of HPCG were preserved.

The project was built using:

- **CMake**
- **GNU g++ Compiler**
- **Ubuntu Linux on Windows Subsystem for Linux (WSL)**

## 8. System Architecture

The experiments were performed on the following hardware:

| **Component** | **Specification** |
|---|---|
| CPU Model | Intel Core i5-5300U @ 2.30 GHz |
| Architecture | x86_64 |
| Sockets | 1 |
| Cores | 2 |
| Threads | 4 |
| L1 Cache | 64 KB (2 instances) |
| L2 Cache | 512 KB (2 instances) |
| L3 Cache | 3 MB |
| RAM | 8 GB |
| Operating System | Ubuntu Linux on WSL |
| Virtualization | Microsoft Hypervisor |

## 9. Experimental Setup

- HPCG Reference Implementation was cloned and built using CMake.
- The input problem size was configured using:

16 × 16 × 16 grid

Runtime: 30 seconds

- Two executions were performed:
    1. **Baseline execution with original SpMV**
    2. **Optimized execution with unrolled SpMV**

All other kernels were kept unchanged to isolate the effect of SpMV optimization.

## 10. Performance Evaluation

## Baseline Results

| **Metric** | **Value** |
|---|---|
| SpMV Time | 24.6904 sec |
| SpMV GFLOP/s | 0.123072 |
| Total Time | 157.837 sec |
| Total GFLOP/s | 0.1307 |

## Optimized Results

| **Metric** | **Value** |
|---|---|
| SpMV Time | 24.4821 sec |
| SpMV GFLOP/s | 0.124118 |
| Total Time | 165.699 sec |
| Total GFLOP/s | 0.124442 |

## Observation

- SpMV kernel performance improved
- Execution time for SpMV decreased
- SpMV GFLOP/s increased
- Total HPCG score slightly decreased, which is acceptable as per project requirements

## 11. Discussion

Although the overall HPCG score decreased slightly, the project objective was to **improve kernel-level performance**, which was successfully achieved. The small improvement confirms that:

- SpMV is highly **memory-bound**
- CPU-only optimizations provide limited but measurable improvement
- Further gains can be achieved using:
    - Data format transformations (SELL, ELL, CSR optimizations)
    - SIMD vectorization
    - Multithreading

This project demonstrates how small kernel-level optimizations can be correctly measured and analyzed.

## 12. Conclusion

In this project, the **Sparse Matrix–Vector Multiplication (SpMV) kernel of the HPCG benchmark was successfully optimized using loop unrolling**. The optimized version showed a measurable improvement in SpMV performance while maintaining correctness and numerical stability.

The project achieved the following learning outcomes:

- Understanding of HPCG internal structure
- Practical experience with kernel-level optimization
- Benchmarking and performance evaluation
- System architecture analysis
- Scientific reporting

Future improvements may include SIMD vectorization, cache tiling, and data structure optimization to further enhance performance.

## 13. HPCG-Level Overall Performance Analysis (Bonus Section)

The High Performance Conjugate Gradient (HPCG) benchmark is designed to represent the full behavior of a real scientific solver rather than measuring the speed of a single kernel. It consists of multiple tightly connected computational kernels, including Sparse Matrix–Vector Multiplication (SpMV), Multigrid (MG), Symmetric Gauss–Seidel (SymGS), Dot Products (DDOT), and Vector Updates (WAXPBY). The total HPCG performance is therefore a combined result of all these kernels rather than a single optimized component.

In this project, only the SpMV kernel was optimized using loop unrolling, while all other kernels remained unchanged. The experimental results clearly show that although the SpMV kernel achieved a slight performance improvement, the overall HPCG performance did not improve and instead decreased slightly. This behavior is expected and can be explained by analyzing the relative contribution of different kernels to the total runtime.

From the experimental results, the approximate kernel timing distribution was as follows:

- **SpMV Time:** ~24.5 seconds
- **Multigrid (MG) Time:** ~136 seconds
- **Other kernels (DDOT, WAXPBY, etc.):** ~5 seconds

This indicates that the **Multigrid kernel dominates the total execution time**, consuming more than 80% of the overall runtime. As a result, even though SpMV was improved, the overall HPCG performance is still limited primarily by MG. This shows that HPCG follows **Amdahl's Law**, which states that improving only a small fraction of a program cannot significantly accelerate the entire application.

Another important factor affecting overall HPCG performance is the balance between **computation and communication**. HPCG includes halo exchange operations, memory-bound sparse operations, and synchronization between different solver stages. These communication and memory-access operations introduce overhead that cannot be eliminated through simple CPU loop optimizations alone. Since SpMV itself is already memory-bound, improvements in arithmetic efficiency alone result in only limited gains.

Furthermore, slight variations in cache behavior, branch prediction, and instruction scheduling caused by the optimized loop can also indirectly influence the performance of other kernels. These secondary effects can explain why the total HPCG runtime increased slightly even though the SpMV kernel became faster.

This bonus analysis demonstrates that **true high-performance optimization must be performed at the full application level**, targeting not just one kernel but multiple dominant components such as Multigrid, SymGS, and communication routines. Optimizing only SpMV is useful for educational demonstration and kernel-level study, but full system-level performance improvement requires coordinated optimization across several major components.

## 14. Selected Research Paper and Critical Review

### Selected Paper: "CVR: Efficient Vectorization of SpMV on x86 Processors".

This paper focuses on improving the performance of sparse matrix–vector multiplication (SpMV) on modern x86 processors by exploiting vectorization and optimized data access patterns. It discusses how to organize sparse data structures and loops so that SIMD instructions and caches are used more efficiently, which directly relates to the kind of kernel-level optimization performed in this project.

The key idea in the paper is that, by restructuring the SpMV kernel and carefully controlling the way nonzero elements are accessed, the processor can perform several multiplications

in parallel and reduce control overhead. This inspired the use of loop unrolling in our implementation, which is a simpler but conceptually similar idea aimed at increasing instruction-level parallelism.

## Strengths of the paper:

- Provides a clear motivation for why SpMV is difficult to optimize.
- Explains how hardware features such as SIMD units and caches can be exploited.
- Presents a systematic approach to modifying the kernel to improve performance.

## Weaknesses or limitations:

- The techniques can be complex to implement in full generality for all matrix structures.
- Some optimizations are highly hardware-specific and may not directly transfer to every platform.
- The focus is on vectorization and specialized formats, which may be harder to maintain in general-purpose codes.

In this project, instead of fully re-implementing the data formats proposed in the paper, a lighter-weight technique—loop unrolling—was implemented. This keeps the code simple and portable while still capturing the core idea of doing more useful work per loop iteration and reducing control overhead.

# 15.Proposed Optimization With Diagrams

## Baseline SpMV Execution

Baseline SpMV (CSR Format)

```
for (j = 0; j < nnz; j++) {
   sum += A[j] * x[col[j]];
}
```

One operation per loop
High loop overhead
Memory-bound

## Way Unrolled SpMV (Optimized)

Optimized SpMV (4-Way Unrolling)

```
for (j = 0; j < nnz; j += 4) {
 sum0 += A[j]   * x[col[j]];
 sum1 += A[j+1] * x[col[j+1]];
 sum2 += A[j+2] * x[col[j+2]];
 sum3 += A[j+3] * x[col[j+3]];
}
sum = sum0 + sum1 + sum2 + sum3;
```

4 operations per loop
Lower overhead
Better CPU usage

## CVR SIMD-Based SpMV Concept